

实验一 4-6 学时实践记录（提交汇编稿）

成员：20241328 蔡贺俊（Alice）、20241304 元泓鉴（Bob）

4-6学时 任务1 OpenSSL 库编程实践（蔡贺俊）

环境：openEuler 24.03 LTS（WSL2）、OpenSSL 3.0.12（EVP API）

代码：code/openssl_sm/sm_demo.c + Makefile

编译

```
$ cd code/openssl_sm && make
gcc -Wall -O2 -o sm_demo sm_demo.c -lcrypto
```

运行

```
$ ./sm_demo
== SM4-CBC (EVP) ==
key = 78da5de71dc1cbe89898b1c2a9fb1198
cipher = e255744fdbe034eb0f2a0ba37b21629d121588017407540a6f87758c8cc125b443a899bd7f0bd93cb6
f397db72bb626
plain = Hello 密码系统设计 20241328 蔡贺俊
SM4 roundtrip: PASS
== SM3 digest / HMAC (EVP) ==
sm3 = e66a3cfe775153beefdbc3ffc5e30a78c0fe2b6fc43891ab9616abdb1962eaa6
hmac-sm3 = dlceec90647b6dbb21f33624a8527b7d96f523bffffed7969e635185f13aeb1a
== SM2 sign/verify, encrypt/decrypt (EVP) ==
siglen = 70
SM2 verify: PASS
session key = 7db0c30b11543718dce0f18604192e68
kc len = 124
SM2 key roundtrip: PASS
```

实现说明

- SM4-CBC: `EVP_sm4_cbc()` + `EVP_CipherInit_ex/Update/Final`, 密钥由 `RAND_bytes` 随机生成 16 字节, 默认 PKCS#7 填充
- SM3: `EVP_DigestInit_ex(EVP_sm3())`; HMAC: `EVP_MAC_fetch("HMAC")` 后用 `OSSL_PARAM` 指定 `digest=SM3`
- SM2 密钥生成: `EVP_PKEY_CTX_new_from_name(NULL, "SM2", NULL)`; 签名验签: `EVP_DigestSign/Verify` 系列配
- SM3; 加密解密: `EVP_PKEY_encrypt/decrypt` (密文为 ASN.1 DER, 124 字节)
- 坑1: 生成 EC 类型密钥挂 SM2 曲线不会走 SM2 算法路径, 必须生成 SM2 类型密钥 (首版 `sign FAIL` 的原因)
- 坑2: `EVP_DigestSignFinal/EVP_PKEY_encrypt` 的 `size_t` 长度参数是 `in/out` 参数, 调用前必须置为缓冲区大小, 传 0 直接失败
- 说明: 此处 SM3 值与任务1 中 `plain.txt` 摘要不同, 因本程序消息串不含末尾换行符

小结

OpenSSL 库编程完成

SM2（加密解密、签名验签）、SM3（摘要、HMAC）、SM4（加密解密）调用，全部往返 PASS。

4-6学时 任务2 GmSSL 库编程实践（蔡贸俊）

环境: openEuler 24.03 LTS (WSL2)、GmSSL 3.3.0-dev.1183 (libgmssl)

代码: code/gmssl_sm/gm_demo.c + Makefile

编译

```
$ cd code/gmssl_sm && make
gcc -Wall -O2 -o gm_demo gm_demo.c -L/usr/local/lib -Wl,-rpath,/usr/local/lib -lgmssl
```

运行

```
$ ./gm_demo
== SM4-CBC (GmSSL lib) ==
key = d81254e31981eaa4dafc361368e10bf8
cipher = 44070f49684163b359df1d818bb664a9ad36a036dd7be2205bc7cc8afbe4a8ca1be36c03e89833c3dfe
fc7e
plain = Hello 密码系统设计 20241328 蔡贸俊
SM4 roundtrip: PASS
== SM3 / SM3-HMAC (GmSSL lib) ==
sm3 = e66a3cfe775153beefdbc3ffc5e30a78c0fe2b6fc43891ab9616abdb1962eaa6
hmac-sm3 = dlceec90647b6dbb21f33624a8527b7d96f523bffffed7969e635185f13aeb1a
== SM2 sign/verify, encrypt/decrypt (GmSSL lib) ==
siglen = 71
SM2 verify: PASS
session key = ffb756a0bf538c1b5ea4a34b61c1327d
kc len = 123
SM2 key roundtrip: PASS
```

实现说明

- SM4-CBC: SM4_CBC_CTX + sm4_cbc_encrypt_init/update/finish (finish 内完成 PKCS#7 填充处理)，解密对称

- SM3: sm3_init/sm3_update/sm3_finish; HMAC: sm3_hmac_init/update/finish

- SM2: sm2_key_generate 生成密钥; 签名 sm2_sign_init/update/finish (ID 取 1234567812345678); 验签 sm2_verify_init/update/finish; 加密解密 sm2_encrypt/sm2_decrypt

- 坑1: GmSSL 3 的摘要收尾函数名为 sm3_finish, 误用 sm3_final 导致隐式声明与 undefined reference

- 坑2: cmake --install 后 libgmssl.so 位于 /usr/local/lib 且不在 ldconfig

缓存中, 首次链接/运行失败; 已写入 /etc/ld.so.conf.d/gmssl.conf 并 ldconfig, Makefile 同时带 -L 与 rpath

- 交叉验证: SM3 与 HMAC-SM3 值与 hours46-1 的 OpenSSL EVP

实现逐字节一致 (e66a3cfe.../dlceec90...), 两套库互验正确

小结

GmSSL 库编程完成 SM2（加密解密、签名验签）、SM3（摘要、HMAC）、SM4（加密解密）调用，全部 PASS，并与 OpenSSL 实现交叉验证一致。

4-6学时 任务3 OpenSSL 库编程数字信封

Alice→Bob（蔡贺俊）

代码: code/envelope_oss1/alice_send.c + Makefile

用法: alice_send <我的私钥pem> <对方公钥pem> <输出目录>

本次运行: alice_send ~/work/task3/alice_openssl_priv.pem

keys/20241304_yuanhongjian/sm2_openssl_pub.pem exchange/task3_lib

运行

```
$ ./code/envelope_oss1/alice_send ~/work/task3/alice_openssl_priv.pem keys/20241304_yuanhongjian/sm2_openssl_pub.pem exchange/task3_lib
k = 83773f00767a2119a90ea6700e9239c4
C len = 32
KC len = 124
S1 len = 72
self verify: PASS
self decrypt: PASS
envelope len = 228, written to exchange/task3_lib
$ ls -l exchange/task3_lib/
C.bin 32 / envelope.bin 228 / KC.bin 124 / lens.txt 30 / S1.bin 72
```

实现说明

- 程序内完成协议全流程: RAND_bytes 生成 16 字节 k → EVP_SM4-CBC 加密明文得 C → EVP_PKEY_encrypt (Bob 公钥) 加密 k 得 KC → EVP_DigestSign (SM3, 我的私钥) 签名 C 得 S1 → 拼接 C||KC||S1 写 exchange/task3_lib/ 并附 lens.txt 长度表
- 发送前自验: 用自己公钥验 S1、用 k 解 C 与明文比对, 双 PASS 才发布
- 坑: PEM_read_PUBKEY 为 4 参数 (fp, cb, u, pass), 漏写第 4 个编译报 too few arguments
- Bob 接收侧与命令版步骤相同 (验签→pkeyutl 解密 KC→sm4 解密 C), 见 MAILBOX 通知

小结

OpenSSL 库编程实现带签名数字信封发送侧, 自验通过, 信封已发布 exchange/task3_lib/ 待 Bob 接收。

4-6学时 任务4 GmSSL 库编程数字信封

Bob→Alice（蔡贺俊, 接收侧）

代码: code/envelope_gmssl/alice_rcv_gm.c + Makefile

用法: alice_rcv_gm <我的GmSSL私钥pem> <口令> <对方GmSSL公钥pem> <信封目录>

接收流程: sm2_verify_init/update/finish 验签 S1 → sm2_decrypt 解 KC 得 k → sm4_cbc_decrypt_init/update/finish 解 C 得 P; iv.bin 存在则用之, 否则全 0 IV。

自测（用自己密钥对模拟发送→接收，验证代码路径与 DER 格式处理）

```
$ gmssl rand -hex -outlen 16 > k.hex; python3 ... < k.hex > k.bin
$ gmssl sm4_cbc -encrypt -pkcs7_padding -key $(cat k.hex) -iv 0000000000000000000000000000
00 -in plain.txt -out C.bin
$ gmssl sm2sign -key alice_gmssl_priv.pem -pass ***** -in C.bin -out S1.bin
$ gmssl sm2encrypt -pubkey alice_gmssl_pub.pem -in k.bin -out KC.bin
$ alice_rcv_gm alice_gmssl_priv.pem ***** alice_gmssl_pub.pem .
note: iv.bin absent, use zero IV
Sm2Verify(PKb, S1): PASS
k len = 16
P = selftest 20241328
```

结论：验签、解密钥、解明文全链路 PASS；CLI 产生的 DER 签名/密文与库函数直接兼容，无需额外转换。

真实交换（待 Bob 库版信封）

等元泓鉴将库版信封（C.bin、KC.bin、S1.bin、iv.bin）发布到 exchange/task4_1ib/ 后，运行：
alice_rcv_gm <我的GmSSL私钥> <口令> keys/20241304_yuanhongjian/sm2_gmssl_pub.pem
exchange/task4_1ib
结果将追加到本文件。

真实交换结果（Bob 库版信封 exchange/task4_1ib/）

```
$ ls exchange/task4_1ib/
C.bin KC.bin S1.bin
$ ./code/envelope_gmssl/alice_rcv_gm ~/work/task3/alice_gmssl_priv.pem ***** keys/20241304
_yuanhongjian/sm2_gmssl_pub.pem exchange/task4_1ib
note: iv.bin absent, use zero IV
Sm2Verify(PKb, S1): PASS
k len = 16
P = 20241304 元泓鉴
```

结论：Bob 库版信封（本次 IV 为全 0）验签、解密钥、解明文全链路 PASS，明文为 Bob 学号姓名，4-6学时任务4 接收侧完成。双向库版数字信封（我发 task3_1ib / 他发 task4_1ib）均已互通成功。